COLUMNCAST: Budgeted Workload-Aware Replication

Pratyoy Das
pratyoyd@uci.edu
University of California
Irvine, California, USA

Vishal Chakraborty
vchakrab@uci.edu
University of California
Irvine, California, USA

Sharad Mehrotra
sharad@ics.uci.edu
University of California
Irvine, California, USA

Abstract

Analytical queries over normalized schemas still pay substantial cost to repeatedly join large fact tables with dimensions. Existing solutions address this either at the storage layer, by materializing fact-side auxiliary state, or at execution time, by constructing and propagating filters on each query. We explore the middle ground: selectively materializing only a small set of high-value dimension attributes under an explicit storage budget, and using them only when they help. We present COLUMNCAST, a storage-layer framework that *grafts* selected dimension attributes onto the foreign-key side of fact tables while preserving the normalized logical schema. A grafted attribute can either eliminate a dimension join or allow a dimension predicate to be applied directly during fact-table access. COLUMNCAST operates in two phases. Offline, it analyzes workloads and selects a budget-feasible set of grafts that maximizes expected utility. Online, a graft-aware rewriter decides whether an available graft should be used for a query. On the Star Schema Benchmark, COLUMNCAST achieves an overall 1.41× speedup with about 8% storage overhead. This corresponds to 76% of SSB++'s performance with 5.5× less storage overhead. These results suggest that selective grafting is a practical new design point between normalized schemas, denormalized layouts, and runtime-only join-reduction techniques.

ACM Reference Format:
Pratyoy Das, Vishal Chakraborty, and Sharad Mehrotra. 2018. COLUMNCAST: Budgeted Workload-Aware Replication. In . ACM, New York, NY, USA, 4 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

The decoupling of compute and storage in modern analytical engines has accelerated the adoption of open table formats such as Apache Iceberg, Hudi, and Delta Lake [2–4]. Analytical queries over normalized schemas still pay substantial cost in repeatedly joining fact tables with dimensions to retrieve selective predicates, grouping keys, and projected attributes needed for execution.

Prior work addresses this problem from two complementary directions. Storage-oriented techniques, such as augmented fact table indexing [10] and join-index-style methods [6, 11], move dimension-derived information closer to the fact table by materializing auxiliary fact-side state and, in some cases, real dimension values [10]. Execution-oriented techniques, such as semijoin-based filtering [17], predicate transfer [16], and related information-passing approaches [8, 12, 14, 19], reduce scan and join cost by constructing and propagating filters at runtime. We compare these in Fig. 1(b).

However, these two lines of work lie at opposite ends of the trade-off space. Storage-oriented approaches pay persistent redundancy up front, but typically do not select replicated attributes under an explicit storage budget and use them unconditionally once materialized. Execution-oriented approaches, in contrast, avoid persistent redundancy but rebuild filters and related runtime state on every query, so their cost cannot be amortized across queries.

COLUMNCAST explores the middle ground. It selectively *grafts* high-value dimension attributes onto the foreign-key side of the fact table, trading bounded storage overhead for reusable runtime savings. A grafted attribute can eliminate a dimension join when the query needs only that attribute, or let a dimension predicate be applied directly during fact-table access. COLUMNCAST is selective in both what it materializes and when it uses it: offline, it selects a budget-feasible set of grafts from workload logs; online, a graft-aware rewriter decides whether to exploit an available graft.

A second aspect of the design is that it decouples normalized query formulation from denormalized execution. Users continue to write queries over the normalized schema, while the engine is free to maintain and exploit denormalized physical state underneath. In this sense, COLUMNCAST is not simply another denormalized layout, but a storage-layer optimization framework that preserves the normalized user-facing abstraction while selectively introducing physical redundancy where it is most beneficial.

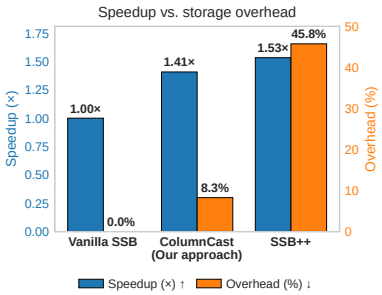
We evaluate COLUMNCAST on the Star Schema Benchmark (SSB) [10]. Under a modest storage budget, the most valuable grafts are a small set of low-cardinality date and geography attributes. As shown in Figure 1(a), these grafts recover much of the benefit of aggressive denormalization at a small fraction of its storage cost. Compared to the vanilla dataset with no grafted columns and to SSB++, which grafts all columns referenced by any query, COLUMNCAST achieves an overall 1.41× speedup with about 8% storage overhead. This corresponds to 76% of SSB++'s performance while incurring 5.5× less storage overhead, and it avoids the query regressions introduced by indiscriminate grafting.

In summary, we:

- introduce COLUMNCAST, a storage-layer framework for selective attribute-level grafting that preserves a normalized logical schema while exploiting denormalized physical state underneath.
- formulate graft selection as a storage-constrained workload optimization problem and design an online graft-aware rewriter that decides when available grafts should be exploited.
- evaluate COLUMNCAST on SSB and show that it recovers much of denormalization's benefit at a fraction of the storage cost.

We conclude with future work in Sec. 3.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX



Technique / Work	Stores fact-side auxiliary state	Materializes real dimension values on fact side	Query-specific fact-side pruning	Chooses what to materialize under a storage budget	Decides at query time whether to use the structure
Yannakakis [17]	✗	✗	✓	✗	✗
Predicate Transfer (PT) [16]	✗	✗	✓	✗	✗
Robust Predicate Transfer (RPT) [18]	✗	✗	✓	✗	✗
PLAQUE [8]	✗	✗	✓	✗	✓
LIP [20]	✗	✗	✓	✗	✓
RPT+ [12]	✗	✗	✓	✗	✓
Dimension-Join [6]	✓	✗	✗	✗	✗
Bitmap / Join Indexes [11]	✓	✗	✗	✗	✗
Parachute [14]	✓	✗	✓	✗	✗
Augmented Fact Table Indexing [10]	✓	✓	✓	✗	✗
ColumnCast (this work)	✓	✓	✓	✓	✓

Figure 1: Positioning and performance of ColumnCast. Left: performance against baselines. Right: comparison with prior work.

Q4.1	$Q_4.1'$ (2 joins eliminated, 1 pred. push down)

`SELECT d_year, c_nation,`
`  sum(lo_revenue -`
`lo_supplycost)`
`  as profit`
`FROM date, customer, supplier,`
`  part, lineorder`
`WHERE lo_custkey = c_custkey`
`AND lo_suppkey = s_suppkey`
`AND lo_partkey = p_partkey`
`AND lo_orderdate = d_datekey`
`AND c_region = 'AMERICA'`
`AND s_region = 'AMERICA'`
`AND (p_mfgr = 'MFGR#1'`
`  OR p_mfgr = 'MFGR#2')`
`GROUP BY d_year, c_nation`
`ORDER BY d_year, c_nation;`
	`SELECT lo_d_year AS d_year,`
	`c_nation,`
	`  SUM(lo_revenue - lo_supplycost)`
	`AS profit`
	`FROM customer, part, lineorder`
	`WHERE lo_custkey = c_custkey`
	`AND lo_partkey = p_partkey`
	`AND lo_c_region = 'AMERICA'`
	`AND lo_s_region = 'AMERICA'`
	`AND (p_mfgr = 'MFGR#1' OR p_mfgr =`
	`'MFGR#2')`
	`GROUP BY lo_d_year, c_nation`
	`ORDER BY lo_d_year, c_nation;`

Figure 2: Grafted dimensions in SSB Query 4.1 (57% speedup).

2 ColumnCast

We begin with an example to formalize the optimization problem (Sub. 2.1), then present our two-phase grafting approach (Sub. 2.2) and preliminary evaluation (Sub. 2.3) on DuckDB[9].

2.1 Example & Problem Formalization

Consider SSB Q 4.1 (Fig. 2). The query joins the fact table `lineorder` with the `date`, `customer`, `supplier`, and `part` dimensions, groups by `d_year` and `c_nation`, and filters on `c_region`, `s_region`, and `p_mfgr`. The query touches only a small number of dimension attributes, several of which are low-cardinality and highly compressible.

Observation 1. If all referenced attributes of a dimension are available in the fact table, that join can be eliminated. For example, if a query only needs `d_year` from date, then grafting `d_year` into `lineorder` removes the date join.

Observation 2. When a join cannot be eliminated, grafting a predicate attribute is still useful. In Q4.1, grafting `c_region` and `s_region` allows these predicates to be evaluated directly during the fact-table scan. This enables efficient zone-map pruning in engines like DuckDB and reduces the data entering the remaining joins. Fig. 2 shows the rewritten query. With just these three grafts (`d_year`, `c_region`, and `s_region`), the database grows by only 5%, while the query speeds up by 57%.

Observation 3. In Q 4.3, the grafted column `d_year` is *not* useful as it materializes an expensive disjunctive predicate on the much larger fact table. Thus, not all grafts improve query performance.

These three observations lead directly to the optimization problem solved by ColumnCast. Let $R_i(q)$ be the set of attributes of dimension D_i referenced by query q, and let $P_i(q) \subseteq R_i(q)$ be the subset used in predicates. For an equally weighted workload W, we define the utility of a G of grafts as $U(G) = \sum_{q \in W} \Delta(q, G)$, where

$$\Delta(q, G) = \sum_i \mathbf{1}[R_i(q) \subseteq G] + \sum_i \big(1 - \mathbf{1}[R_i(q) \subseteq G]\big) \cdot |P_i(q) \cap G|,$$

The first term counts join eliminations and the second counts predicate pushdowns for dimensions whose joins remain.

Each graft also incurs storage. We model this with a normalized weight for an attribute defined as: $w(a) = \frac{|F|}{S(F)} \frac{\lceil \log_2 |(a)| \rceil}{8} + \mu_{(a)}$.

Therefore, we have the following optimization problem:

$$G^* = \arg\max_{G \subseteq C} U(G) \quad \text{s.t.} \sum_{a \in G} w(a) \leq \beta. \tag{1}$$

Here, $|F|$ is the fact-table cardinality, $S(F)$ is the base fact-table size, $\mu_{(a)}$ captures residual storage-engine overheads such as metadata and compression overheads, and β is the normalized storage budget.

For SSB at SF300, $|F| = 1.8 \times 10^9$. Using DuckDB storage statistics with block size 262,144 bytes and 242,262 used blocks, the fact-table footprint is $S(F) = 242{,}262 \times 262{,}144 \approx 63.51 \times 10^9$ bytes (≈ 41.85 GiB). This gives $\frac{|F|}{S(F)} \approx \frac{1.8 \times 10^9}{41.85 \times 10^9} \approx 0.0430$. We distinguish three classes of attributes based on their cardinality and metadata: Class 1 for `p_mfgr`, `d_year`, `s_region`, and `c_region`, with $\mu_1 = 0.05$; Class 2 for `d_yearmonth`, `s_nation`, `c_nation`, and `p_category`, with $\mu_2 = 0.10$; and Class 3 for `s_city`, `c_city`, and `p_brand1`, with $\mu_3 = 0.20$. We show the estimated weights for the SSB graft candidates in Table 1. Table 1.

2.2 Implementation: Two-Phase Grafting

ColumnCast has two phases: the offline phase decides *what* to graft; the online phase decides *when* to use it. DuckDB still chooses the final physical plan.

Offline phase. The Column Grafter enumerates candidate attributes over PK-FK joins, estimates their storage cost and workload utility, and selects a set under a budget. The subsequent materialization uses standard `ALTER TABLE`. In our current prototype, any non-key attribute on the primary-key side is a valid candidate.

Storage is estimated using the normalized weight $w(a)$ from the previous subsection. A graft set G is feasible if $\sum_{a \in G} w(a) \leq \beta$.

Utility is estimated from the same two effects used in our formalization: join elimination and predicate pushdown. A candidate is useful if it helps fully cover a dimension and remove a join, or if it enables selective predicate pushdown when the join remains. Low-cardinality filter attributes are therefore strong candidates, while high-cardinality projection-only attributes are usually poor ones.

Attribute	Domain size	Bits	Class	Per-row bytes = $\frac{\#\text{bits}}{8} + \mu$	$w(a) = 0.0430\times$ per-row bytes
d_year	7	3	1	0.375 + 0.05 = 0.425	0.0183
d_yearmonth	84	7	2	0.875 + 0.10 = 0.975	0.0419
s_region	5	3	1	0.375 + 0.05 = 0.425	0.0183
c_region	5	3	1	0.375 + 0.05 = 0.425	0.0183
s_nation	25	5	2	0.625 + 0.10 = 0.725	0.0312
c_nation	25	5	2	0.625 + 0.10 = 0.725	0.0312
s_city	250	8	3	1.000 + 0.20 = 1.200	0.0516
c_city	250	8	3	1.000 + 0.20 = 1.200	0.0516
p_mfgr	5	3	1	0.375 + 0.05 = 0.425	0.0183
p_category	25	5	2	0.625 + 0.10 = 0.725	0.0312
p_brand1	1000	10	3	1.250 + 0.20 = 1.450	0.0624

Table 1: Estimated normalized storage weights for SSB graft candidates under the refined storage model.

Online phase. Currently the online query rewriter is separate from the query optimizer. It rewrites the queries heuristically based on the available grafts and passes it to DuckDB to optimize the query. If the grafted attributes fully cover a dimension, the join is eliminated. Else the rewriter uses grafted columns only for highly selective point or range predicates, and only for simple or conjunctive predicates.

2.3 Preliminary Evaluation

We evaluate how COLUMNCAST incrementally grafts promising attributes into `lineorder` under a budget overhead of 10%. All experiments were run in DuckDB (v1.4.4) on a server with 4× Intel Xeon E5-4640 v2 CPUs (40 physical cores, 80 hardware threads) at 2.20 GHz, 125 GiB RAM, and a 7.68 TB Samsung PM893 SATA SSD. We report the median of runs 2–11 (11 total) to exclude warm-up. DuckDB already supports min–max filter transfer from the hash-table build side to the probe side on join keys [12], as well as multiple compression techniques [13]. These features make newly added grafts both effective and inexpensive: they help reduce scan costs at query time while keeping the storage overhead low.

Our offline analysis identifies `d_year` and `s_region` as the most valuable early grafts, followed by `c_region` and `s_nation`. Together, these four grafts improve 10 of the 13 SSB queries, with speedups ranging from 10% to 77%, while causing no meaningful slowdown on the remaining queries. Fig. 3 summarizes the resulting speed–storage tradeoff as grafts are added incrementally. Table 2 shows how those gains are distributed across queries. `s_region` is the main driver of the Q2 family, `d_year` dominates Q1.1, and `c_region` contributes most strongly to Q4.1 and Q4.2, while more complex queries such as Q3.1 benefit from multiple grafts together. Notably, the earlier grafts contribute to more queries than the later ones, which is consistent with the ranking produced by our offline analysis. Overall, the analysis confirms that most of the observed speedup comes from a small number of carefully chosen grafts.

TPC-H. We evaluate COLUMNCAST on TPC-H Q9 (a challenging query on DuckDB [7]). We graft `r_name` into `supplier`, `n_name` into `customer`, and `c_custkey` and `o_orderdate` into `lineitem`. This yields a 51% speedup at 11% storage overhead, consistent with the SSB results. This is a promising result, although a broader TPC-H evaluation remains future work.

3 Conclusion and Future Work

We introduced COLUMNCAST, a lightweight approach for reducing join cost through selective grafting. Our early results are encouraging

Query	d_year	s_region	c_region	s_nation	Overall
1.1	35.70%	–	–	–	**35.70%**
1.2	–	–	–	–	2.94%
1.3	–	–	–	–	0.36%
2.1	11.05%	58.98%	–	–	**76.55%**
2.2	12.81%	33.14%	–	–	**50.19%**
2.3	13.01%	28.26%	–	–	**44.94%**
3.1	18.26%	11.27%	2.33%	10.93%	**49.36%**
3.2	2.07%	–	–	36.59%	**39.42%**
3.3	10.70%	–	–	–	**10.70%**
3.4	–	–	–	–	1.72%
4.1	7.76%	18.01%	21.23%	–	**57.38%**
4.2	–	14.25%	42.87%	–	**63.23%**
4.3	–	–	23.63%	–	**23.63%**

Table 2: Speedup attribution by grafted attribute.

and point to a broader systems question: how should a database expose and optimize selective denormalization as a design choice?

Optimizer integration. In COLUMNCAST, graft selection and query rewriting are external to the optimizer. A graft-aware cost-based optimizer could reason about equivalences between base and grafted attributes, eliminate redundant PK–FK joins, choose between fact-side and dimension-side predicate evaluation, and enable rewrites such as aggregation pushdown. Grafting changes more than access paths: it changes both the optimizer's logical opportunities and the underlying physical tradeoffs. This suggests a joint optimization problem over query plans and storage layout. A generalized formulation would jointly optimize which attributes to graft, where to store them, how to compress them, and when to retire them.

Incremental grafts. Grafts must remain correct and useful under inserts, deletes, and updates to both fact and dimension tables. This raises questions of incremental maintenance cost, consistency guarantees, and when stale grafts should be recomputed or invalidated. Their value is also workload-dependent: previously useful grafts may become dead weight, while new hot predicates may justify new

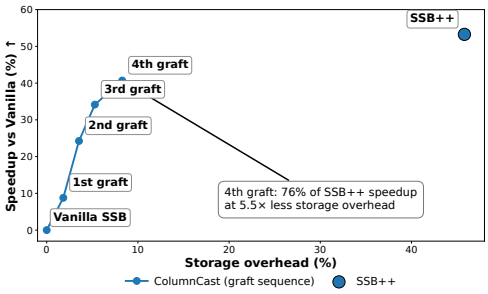


Figure 3: Speed-storage tradeoff on SSB (SF=300).

grafts. Thus, how to detect workload drift and adapt the graft set without oscillation or excessive reorganization remains open.

Generalizing across workloads and schemas. Our current setting focuses on one-to-many PK–FK joins, where the semantics are clean and join-elimination benefits are easy to isolate. Many real workloads, however, contain bridge tables and many-to-many joins. In such settings, grafting may still reduce join work, but it also risks larger storage blowup and more complex maintenance. The key question is not only whether a graft removes a join, but when the extra denormalization remains worthwhile.

Moreover, COLUMNCAST is complementary to SIP-style techniques such as Plaque[8], Parachute[14], and RPT+[12]. Their runtime behavior can, in principle, provide feedback about which predicates and attributes are repeatedly worth pushing toward the fact side, thereby informing which grafts should be materialized. These interactions may be even more important in distributed engines, where eliminating dimension joins may save not only local CPU work but also network and synchronization cost, while increasing replication and maintenance overhead. Studying this tradeoff in systems such as AsterixDB[1] and SparkSQL[5] is an important next step.

Finally, COLUMNCAST suggests a broader role in self-driving database systems [15]. Instead of treating grafting as a one-off tuning step, a self-driving system could continuously decide when to create, retain, or remove grafts under explicit storage and maintenance budgets. This raises questions about observability, stability, and control for workload-adaptive storage design, and suggests that grafting may serve as a new actuator for autonomous physical design.

References

[1] S. Alsubaiee, Y. Altowim, H. Altwaijry, A. Behm, V. R. Borkar, Y. Bu, M. J. Carey, I. Cetindil, M. Cheelangi, K. Faraaz, E. Gabrielova, R. Grover, Z. Heilbron, Y. Kim, C. Li, G. Li, J. M. Ok, N. Onose, P. Pirzadeh, V. J. Tsotras, R. Vernica, J. Wen, and T. Westmann. AsterixDB: A scalable, open source BDMS. *Proc. VLDB Endow.*, 7(14):1905–1916, 2014.

[2] Apache Hudi Project. Apache hudi: Upserts, deletes and incremental processing on data lakes. https://hudi.apache.org/, 2026. Accessed 2026-03-08.

[3] Apache Iceberg Project. Apache iceberg table format specification. https://iceberg.apache.org/spec/, 2026. Accessed 2026-03-08.

[4] M. Armbrust, T. Das, L. Sun, B. Yavuz, S. Zhu, M. Murthy, J. Torres, H. van Hovell, A. Ionescu, A. Łuszczak, M. Šešek, F. Xu, et al. Delta lake: High-performance ACID table storage over cloud object stores. In *Proceedings of the VLDB Endowment (PVLDB)*, volume 13, pages 3411–3424, 2020.

[5] M. Armbrust, R. S. Xin, C. Lian, Y. Huai, D. Liu, J. K. Bradley, X. Meng, T. Kaftan, M. J. Franklin, A. Ghodsi, and M. Zaharia. Spark sql: Relational data processing in spark. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, SIGMOD '15, page 1383–1394, New York, NY, USA, 2015. Association for Computing Machinery.

[6] P. Bizarro and H. Madeira. The dimension-join: A new index for data warehouses. In *Proceedings of the 15th Brazilian Symposium on Databases (SBBD)*, 2001.

[7] M. Dreseler, M. Boissier, T. Rabl, and M. Uflacker. Quantifying TPC-H choke points and their optimizations. *Proc. VLDB Endow.*, 13(8):1206–1220, 2020.

[8] Y. Lin and S. Mehrotra. Plaque: Automated predicate learning at query time. *Proc. ACM Manag. Data*, 2(1), Mar. 2024.

[9] H. Mühleisen and M. Raasveldt. DuckDB. https://duckdb.org, 2026. Version 1.4.4.

[10] P. E. O'Neil, E. J. O'Neil, X. Chen, and S. Revilak. The star schema benchmark and augmented fact table indexing. In *Performance Evaluation and Benchmarking: First TPC Technology Conference (TPCTC), Revised Selected Papers*, pages 237–252. Springer, 2009.

[11] Oracle Corporation. *Oracle Database Data Warehousing Guide*, 10g release 1 (10.1) edition, 2003. See discussion of bitmap join indexes.

[12] Y. Qiao, P. Boncz, and H. Zhang. Robust predicate transfer with dynamic execution. *Proceedings of the VLDB Endowment (PVLDB)*, 19(6):1278–1290, 2026.

[13] M. Raasveldt. Lightweight compression in duckdb, Oct. 2022. DuckDB Blog.

[14] M. Stoian, A. Zimmerer, S. Krid, A. L. Ngom, J. Ding, T. Kraska, and A. Kipf. Parachute: Single-pass bi-directional information passing. *Proc. VLDB Endow.*, 18(10):3299–3311, June 2025.

[15] D. Van Aken, A. Pavlo, G. J. Gordon, and B. Zhang. Automatic database management system tuning through large-scale machine learning. In *Proceedings of the 2017 ACM International Conference on Management of Data*, SIGMOD '17, page 1009–1024, New York, NY, USA, 2017. Association for Computing Machinery.

[16] Y. Yang, H. Zhao, X. Yu, and P. Koutris. Predicate transfer: Efficient pre-filtering on multi-join queries, 2025.

[17] M. Yannakakis. Algorithms for acyclic database schemes. In *Proceedings of the 7th International Conference on Very Large Data Bases (VLDB '81)*, pages 82–94, 1981.

[18] J. Zhao, K. Su, Y. Yang, X. Yu, P. Koutris, and H. Zhang. Debunking the myth of join ordering: Toward robust SQL analytics. *Proceedings of the ACM on Management of Data (PACMMOD)*, 2025.

[19] J. Zhao, K. Su, Y. Yang, X. Yu, P. Koutris, and H. Zhang. Debunking the myth of join ordering: Toward robust sql analytics, 2025.

[20] J. Zhu, N. Potti, S. Saurabh, and J. M. Patel. Looking ahead makes query plans robust: making the initial case with in-memory star schema data warehouse workloads. *Proc. VLDB Endow.*, 10(8):889–900, Apr. 2017.